

Ex72_FirWienerPrediction

April 19, 2026

1 MSE StatDig : Chap 7 “Optimum filter”

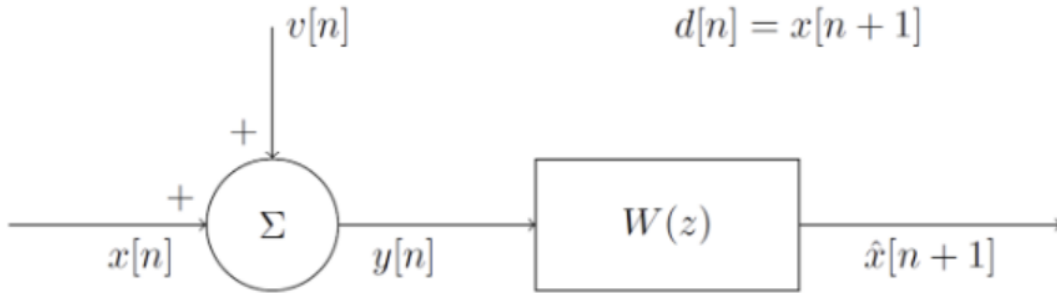
1.1 Ex 7.2 FIR Wiener Prediction

ver : DLY/19.04.2026

1.2 General

1.2.1 Description

Let's consider the linear prediction for a random process AR(1) measured with and without noise.



Let $d[n]$ be an AR(1) process excited by a zero mean white noise with unit variance:

$$x[n] = a[1]x[n-1] + w[n]$$

The goal is to design a filter $W(z)$ which will predict the next sample $x[n+1]$ by observing $y[n]$ where $y[n]$ is the noisy version of $x[n]$.

$$y[n] = x[n] + v[n]$$

where $v[n]$ is a white noise disturbance with variance σ_v^2 .

1.2.2 Work

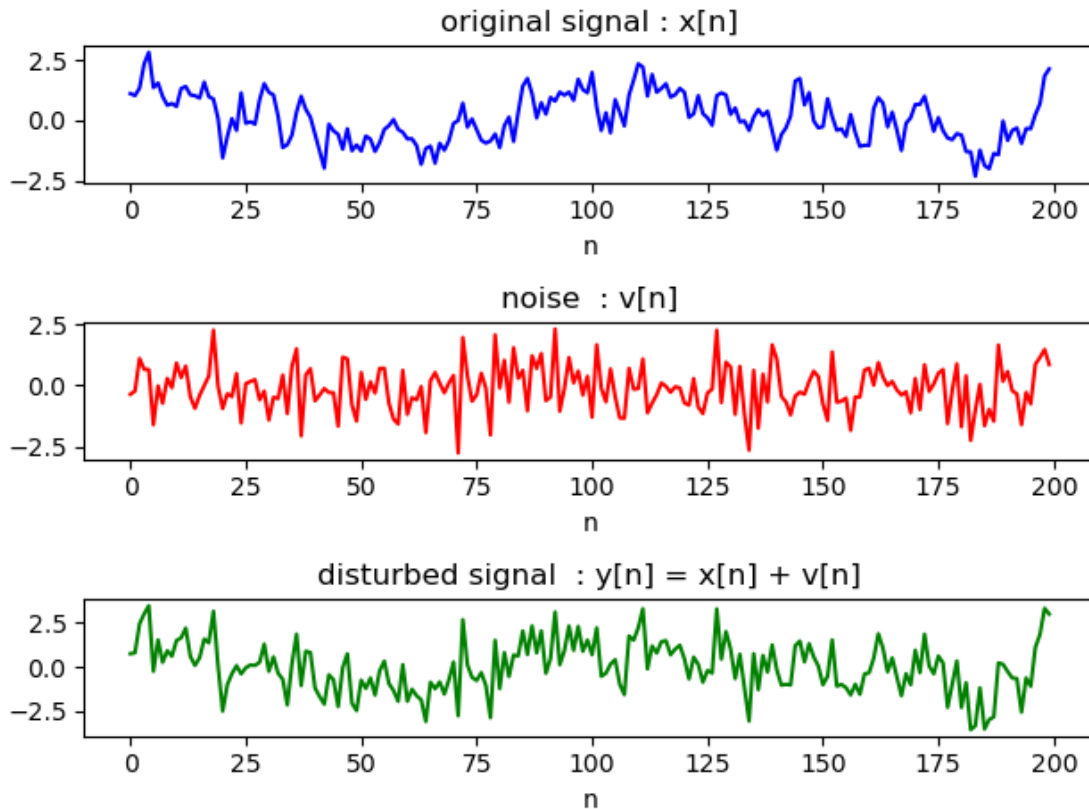
Ex1 : Signal generation

- Display the signals created by the following code :

```

sigmaW = 1; # std of the noise process
sigmaV = 1 # std of the noise perturbation
N = 200
p = 2 #number of coefficient of the filter
worder = p-1 # filter order
a1 = 0.7
maxLag = 10
# fix the random seed for reproducibility
np.random.seed(0)
wn = sigmaW*np.random.normal(0,1,N) # noise process
vn = sigmaV*np.random.normal(0,1,N) # measurement noise
x = signal.lfilter(np.array([1]),np.array([1,-a1]),wn) #AR(1) process
x = x/np.sqrt(np.mean (x**2)) # Noramlize the power to 1
y = x+vn

```



Ex2 : Theoretical Wiener predictor

Ex2.1 Design predictor The **Wiener-Hopf** equation for the linear predictor is:

$$R_y w = r_y(\alpha) \Rightarrow (R_x + \sigma_v^2 I) = r_y(\alpha)$$

As the noise is white, so the autocorrelation is null for $k \neq 0$, the autocorrelation vector $r_y(\alpha)$ is:

$$r_y(\alpha) = r_{xx}(\alpha) + \sigma_v^2 \delta(\alpha)$$

If $\alpha > 0$ which is the case for a prediction, then the autocorrelation vector is given by:

$$r_y(\alpha) = r_x(\alpha)$$

So, the Wiener-Hopf equation becomes:

$$R_y w = r_x(\alpha)$$

The vector $r_x(\alpha)$ is the autocorrelation vector of the AR(1) process, which is given by:

$$r_x(\alpha) = \begin{bmatrix} r_{xx}[\alpha] \\ \vdots \\ r_{xx}[\alpha + p - 1] \end{bmatrix}$$

So, the autocorrelation matrix is for ($p = 2$) and ($a[1] = 0.7$):

$$R_y = \begin{bmatrix} r_{xx}[0] + \sigma_v^2 & r_{xx}[1] \\ r_{xx}[1] & r_{xx}[0] + \sigma_v^2 \end{bmatrix} = \begin{bmatrix} 1 + \sigma_v^2 & a[1] \\ a[1] & 1 + \sigma_v^2 \end{bmatrix} = \begin{bmatrix} 1 + \sigma_v^2 & 0.7 \\ 0.7 & 1 + \sigma_v^2 \end{bmatrix}$$

And the shifted autocorrelation vector for a one-step prediction ($\alpha = 1$):

$$r_x(\alpha) = \begin{bmatrix} r_{xx}[\alpha] \\ r_{xx}[\alpha + 1] \end{bmatrix} = \begin{bmatrix} r_{xx}[1] \\ r_{xx}[2] \end{bmatrix} = \begin{bmatrix} a[1] \\ (a[1])^2 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.49 \end{bmatrix}$$

Finally, the solution of the **Wiener-Hopf** is:

$$w = \begin{bmatrix} 1 + \sigma_v^2 & a[1] \\ a[1] & 1 + \sigma_v^2 \end{bmatrix}^{-1} \cdot \begin{bmatrix} a[1] \\ (a[1])^2 \end{bmatrix} = \begin{bmatrix} 1 + \sigma_v^2 & 0.7 \\ 0.7 & 1 + \sigma_v^2 \end{bmatrix}^{-1} \begin{bmatrix} 0.7 \\ 0.49 \end{bmatrix} = \begin{bmatrix} 0.3114 \\ 0.1796 \end{bmatrix}$$

If the noise is null ($\sigma_v^2 = 0$), then the coefficients become:

$$w = \begin{bmatrix} 1 & a[1] \\ a[1] & 1 \end{bmatrix}^{-1} \cdot \begin{bmatrix} a[1] \\ (a[1])^2 \end{bmatrix} = \begin{bmatrix} 1 & 0.7 \\ 0.7 & 1 \end{bmatrix}^{-1} \begin{bmatrix} 0.7 \\ 0.49 \end{bmatrix} = \begin{bmatrix} a[1] \\ (a[1])^2 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0 \end{bmatrix}$$

The solution is logical. As there isn't any noise, the best coefficient to predict the next sample is the coefficient of the AR process itself:

$$x[n + 1] = a[1] \cdot x[n]$$

Your work:

- Compute the theoretical predictor by implementing a function `wiener_predict_theo()`:

```

“python def wiener_predict_theo(sigmay,a1,order,step): # computes the theoretical Wiener FIR
predictor for an AR(1) process disturbed # by a random noise described by sigmay # Input : # -
sigmay : (scalar) standrad deviation of the noise # - a : (scalar) coefficient of the AR(1) # - order
: (scalar) order of the Wiener FIR filter (order = p-1 => p = order + 1) # - step : (scalar) step
of prediction # Output : # - w : (p,) coefficient of the Wiener FIR predictor

```

```

# -- your code --

```

```

return w

```

```

FIR predictor theoretical step = 1

```

```

w= [0.3011396  0.13960114]

```

```

E= 0.721

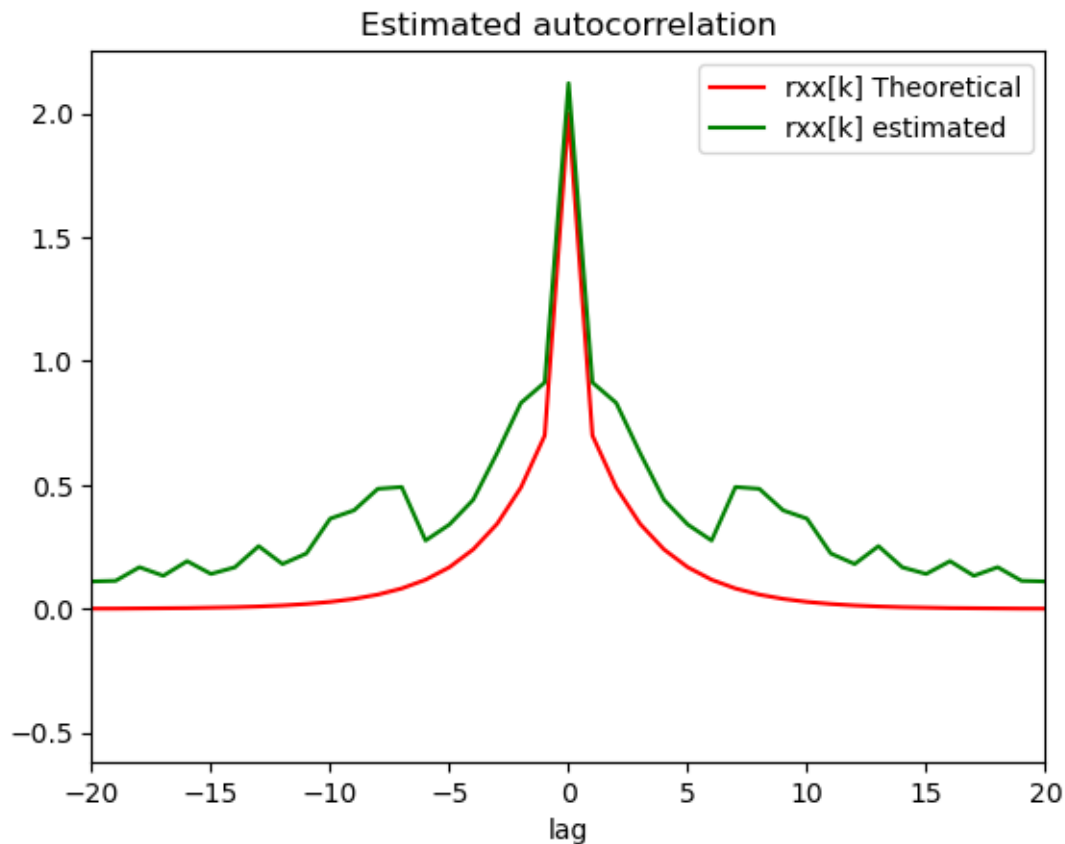
```

Ex2 : Estimated Wiener filter predictor

Ex2.1 : Estimated autocorrelation Unfortunately, in most of the application, the autocorrelation must be estimated.

$$r_y[k] = \frac{1}{N} \sum_{n=0}^{N-1-k} y[n]y[n+k]$$

- Display the estimated autocorrelation $r_y[k]$ and compare it to the theoretical one.



Ex2.2 : Design filter predictor By using the estimated autocorrelation, the Wiener filter coefficient can be estimated. The results shows very close results with the theoretical coefficient because the estimator of the autocorrelation is good for small order (order=p-1). The wiener predictor is given by:

$$w = R_y^{-1} r_y(\alpha)$$

Where R_y is the autocorrelation matrix and $r_y(\alpha)$ is the shifted autocorrelation vector. The autocorrelation matrix is given by:

$$R_y = \begin{bmatrix} r_{yy}[0] & r_{yy}[1] \\ r_{yy}[1] & r_{yy}[0] \end{bmatrix}$$

And the shifted autocorrelation vector for a one-step prediction ($\alpha = 1$):

$$r_y(\alpha) = \begin{bmatrix} r_{yy}[\alpha] \\ r_{yy}[\alpha + 1] \end{bmatrix}$$

And the mean square error (MSE) is given by:

$$\xi_{\min} = r_{yy}[0] - w^T r_y(\alpha)$$

Unfortunately, the mean square error is including the noise variance $\sigma_v^2 = 1$ because $r_{yy}[0] = r_{xx}[0] + r_{vv}[0]$. So the MSE estimated is higher than the theoretical one which was using $r_{xx}[0]$. If you want to remove the noise variance $r_{vv}[0] = \sigma_v^2$, you can use the following formula:

$$\xi_{\min,x} = \xi_{\min} - \sigma_v^2$$

Your work:

- Compute the theoretical predictor by implementing a function `wiener_predict()`:

```
def wiener_predict(y,order,step):
    # computes the estimated Wiener FIR predictor for an AR(1) process disturbed
    # by a random noise described by sigma_v
    # Input :
    # - y      : (N,) noisy signal to be predicted
    # - order  : (scalar) order of the Wiener FIR filter (order = p-1 => p = order + 1)
    # - step   : (scalar) step of prediction (alpha)
    # Output :
    # - w      : (p,) coefficients of the Wiener FIR predictor
    # - E      : (scalar) MSE of the predictor
    # -- your code --

    return w, E
```

```
FIR predictor estimation step = 1
w= [0.32161912 0.25323633]
MSE Ey= 1.618
MSE Ex= 0.618
```

Ex2.3 : Prediction The prediction is given by the following formula:

$$\hat{x}[n+1] = w^T y[n]$$

This can be simply realized by the filter function from scipy `xhat = filter(w,1,y)`.

As the output is the prediction, we can compare the prediction to the original signal $x[n]$ by computing the error and the corresponding MSE

$$e[n] = x[n+1] - \hat{x}[n+1]$$

$$mse = \frac{1}{N} \sum_{n=0}^{N-1} e[n]^2$$

your work : - compute and display the prediction, the error and the MSE

MSE = 0.5416

